

On-Chain Drift Detection for Delegated LLM Trading Agents: A Layered ASI Monitoring and Contract Enforcement Framework

Dogukan Ali Gundogan
Independent Research

dogukangundogan5@gmail.com

Abstract—Delegated AI trading agents are selected on the basis of historical behavioral profiles, yet large language model (LLM)-based agents demonstrably degrade and silently shift strategy over extended deployments. No prior on-chain mechanism exists to detect when a delegated agent has drifted from the mandate that earned its reputation, leaving delegator capital exposed to strategies never explicitly approved. We present the first system to couple continuous per-turn LLM behavioral drift detection with hard pre-execution mandate blocking for delegated trading agents, and to log all compliance events as tamper-evident records on a public blockchain. Concretely, we compute a 12-dimensional *Agent Stability Index* (ASI) each turn, feed it into an ADWIN-based online detector, hard-block mandate violations before they execute, and write every drift and violation event to *Kite Chain* [11]—an AI-native EVM-compatible Layer-1 blockchain—so that delegators can verify the agent’s full compliance history without trusting the monitoring service. Evaluated against 500 simulated market-making sessions with controlled drift injections at three severity offsets (turns 50, 100, and 200), the system achieves a mean detection lag of 0.82 turns, zero false positives, zero false negatives, and a 100% contract violation blocking rate. Post-remediation ASI recovery deviates only 1.7% from the pre-drift baseline. An ablation study over the four ASI metric groups identifies response consistency (Group A, cosine embedding similarity) as the primary driver of fast detection; removing it increases mean detection lag by 0.89 turns (99.4%), while the three remaining groups reduce false positives when combined. All 107 unit and integration tests pass. These results represent a substantive advance over purely observational drift-detection baselines and introduce verifiable, delegator-queryable mandate compliance as a first-class system property.

Index Terms—LLM agents, behavioral drift detection, on-chain monitoring, agent stability index, contract enforcement, ADWIN

I. INTRODUCTION

The emergence of LLM-based autonomous trading agents has introduced a novel principal agent problem: a delegator selects an agent on the basis of its demonstrated strategy. For example, the agent may pursue low-volatility market making with bounded drawdown, yet that strategy may silently erode over time through what Rath [1] terms *agent drift*. Behavioral stability has been shown to drop substantially without explicit long-term memory mechanisms, meaning that the agent ex-

ecuting trades today may bear little resemblance, in strategy terms, to the agent that earned the delegator’s trust last month.

Existing monitoring approaches address distributional shift in financial time series [2] or multi-agent anomaly detection [3], but neither provides (a) mandate-anchored behavioral comparison, (b) hard enforcement that blocks off-mandate actions *before* they execute, nor (c) a cryptographically verifiable audit trail that delegators can query on-chain without trusting the monitoring service itself. The gap between observational drift quantification and actionable capital protection is precisely the problem this work addresses.

We make the following contributions:

- **First coupled detection–enforcement loop for delegated LLM trading agents.** Prior systems [1], [3] detect drift observationally but cannot block off-mandate execution. We close this gap: a YAML-specified mandate is evaluated *before* every tool call, achieving a 100% violation blocking rate across 500 benchmark sessions (§IV-D, Table I).
- **First on-chain, delegator-queryable LLM agent compliance audit.** No prior work provides tamper-evident per-turn compliance records on a public blockchain. We deploy a `DriftRegistry` smart contract on Kite Chain [11] so that delegators can call `verifyRecord(sessionId, payloadHash)` without trusting the monitoring operator (§IV-G).
- **Mandate-anchored, continuously computed ASI for trading agents.** Rath [1] measures ASI retrospectively and without a formal mandate; concept-drift detectors [2] operate on raw financial series, not agent behavioral embeddings. We compute a 12-dimensional ASI every turn against a profiled mandate baseline, reaching a mean detection lag of 0.82 turns versus the hundreds of samples required by window-based financial detectors (§IV-B, §IV-C).
- **Empirical metric-group importance ranking for trading-agent drift.** Through a 12-variant ablation over 500 sessions each, we establish for the first time that (a) embedding cosine similarity (Group A) is the dom-

inant fast-detection signal—removing it increases mean lag by +99.4%—and (b) tool-usage features (Group B) are the primary precision stabilizer—removing them raises FPR from 0.000 to 0.020 without improving speed (§IV-I, Table II).

- **Production deployment on Kite Chain, the first AI-native L1 blockchain.** Existing on-chain AI governance proposals [8] remain architectural; we provide a working end-to-end implementation with measured API latency (p95 18.6 ms) deployed on a chain purpose-built for the agentic economy [11] (§IV-G, §IV-H).

II. RELATED WORK

a) *Agent drift quantification.*: Rath [1] introduces the first systematic quantification of behavioral degradation in multi-agent LLM systems over extended interactions. The work demonstrates that behavioral stability scores decay without explicit long-term memory and proposes the ASI metric family that we adopt and extend. Critically, Rath measures ASI *retrospectively* on completed traces and without a mandate against which drift is defined; our system computes ASI *continuously* each turn relative to a profiled mandate baseline and triggers enforcement before the next tool call executes. Yang et al. [7] study how evolving episodic memory contributes to behavioral drift in long-horizon agents; our episodic consolidation layer (§IV) directly addresses this mechanism.

b) *LLM agent safety and mandate enforcement.*: AgentSpec [5] defines a runtime enforcement framework for LLM agents using trigger–predicate–action rules evaluated before each action, achieving over 90% blocking effectiveness on safety-critical code-execution and driving tasks. Bai et al. [6] propose Constitutional AI, which instills behavioral constraints at training time via self-critique and RLHF. Our work differs in two key respects: (1) we target financial trading mandates whose violation can cause immediate capital loss, requiring *hard* pre-execution blocking at inference time rather than training-time alignment; and (2) we couple enforcement with a real-time drift signal so that the system adapts its sensitivity as the agent deviates from its profiled baseline—a property absent from both AgentSpec and Constitutional AI.

c) *Financial concept drift detection.*: Neri [2] demonstrates that domain-specific concept drift detectors outperform generic statistical methods on financial time series. The paper evaluates ADWIN, DDM, and EDDM variants with domain-tuned feature engineering. We incorporate ADWIN as the online detection backbone but layer mandate-specific behavioral features (the ASI dimensions) over generic distributional statistics, obtaining sub-turn detection lag versus the hundreds of samples typically required by window-based detectors on raw financial series.

d) *Multi-agent anomaly detection.*: He et al. [3] introduce SentinelAgent, which applies graph-based trajectory analysis to detect anomalies in multi-agent systems. SentinelAgent operates at the graph level, making it well suited to coordinated misbehavior, but is purely observational: it does not block off-mandate actions and provides no audit trail

queryable by delegators. Our work is complementary, focusing on a single delegated agent’s mandate adherence and adding hard enforcement and on-chain auditability.

e) *Concept drift surveys.*: Gama et al. [4] provide a comprehensive survey of concept drift adaptation methods for data streams, covering both detection algorithms (ADWIN, DDM, Page-Hinkley) and adaptation strategies. Our work applies streaming drift detection in the novel context of LLM behavioral monitoring, where the “stream” is a sequence of agent-turn ASI scores rather than raw input-feature distributions.

f) *Principal–agent problems in AI delegation.*: The principal–agent problem [9]—in which an agent acts on a principal’s behalf but may pursue diverging interests—is the economic foundation for delegated AI trading. Recent legal and policy analysis [10] identifies the absence of verifiable compliance mechanisms as the primary governance gap in agentic AI systems. Our system operationalizes a technical solution: a continuously monitored mandate whose compliance history is recorded on-chain and independently auditable by the delegating principal.

g) *On-chain AI governance.*: Weng et al. [8] propose a blockchain-monitored agentic AI architecture that records high-level agent decisions on-chain for post-hoc auditing. Kite Chain [11] provides an EVM-compatible Layer-1 blockchain with native identity, payment, and governance primitives designed specifically for autonomous AI agents. Our work advances this line by logging not only decisions but *per-turn behavioral metrics* (ASI scores, KL divergence, mandate violation payloads) as keccak256-hashed records, and by providing a `verifyRecord` contract function that lets delegators confirm compliance without trusting the monitoring operator.

III. PROBLEM FORMULATION

Let $\mathcal{A}$ denote a delegated LLM trading agent operating under mandate $\mathcal{M}$, specified as a set of rules $\{r_1, r_2, \dots, r_k\}$ over observable agent outputs (e.g., position size limits, approved strategy identifiers, maximum drawdown thresholds, prohibited asset classes). At each discrete turn t , the agent produces an output tuple $o_t = (y_t, \tau_t, \rho_t)$ where y_t is the natural language response, τ_t is the tool-call sequence, and ρ_t is the reasoning chain trace.

During a warm-up phase of $N = 50$ turns, a reference distribution p_0 is estimated from the embedding vectors $\{e_1, \dots, e_N\}$, tool-call histograms, and response statistics. The *Agent Stability Index* at turn t is:

$$\text{ASI}_t = w_{\text{RC}} s_{\text{RC},t} + w_{\text{TU}} s_{\text{TU},t} + w_{\text{IC}} s_{\text{IC},t} + w_{\text{BB}} s_{\text{BB},t} \quad (1)$$

where the category weights $(w_{\text{RC}}, w_{\text{TU}}, w_{\text{IC}}, w_{\text{BB}}) = (0.30, 0.25, 0.25, 0.20)$ follow Rath [1], and each $s_{\cdot,t} \in [0, 1]$ is a normalized aggregate of the sub-metrics within that category.

Drift detection is formulated as a change-point problem on the ASI stream $\{\text{ASI}_t\}_{t \geq N+1}$. A *DriftEvent* with severity $\sigma \in \{\text{WARNING}, \text{CRITICAL}\}$ is emitted when ADWIN identifies a statistically significant window shrinkage, or when

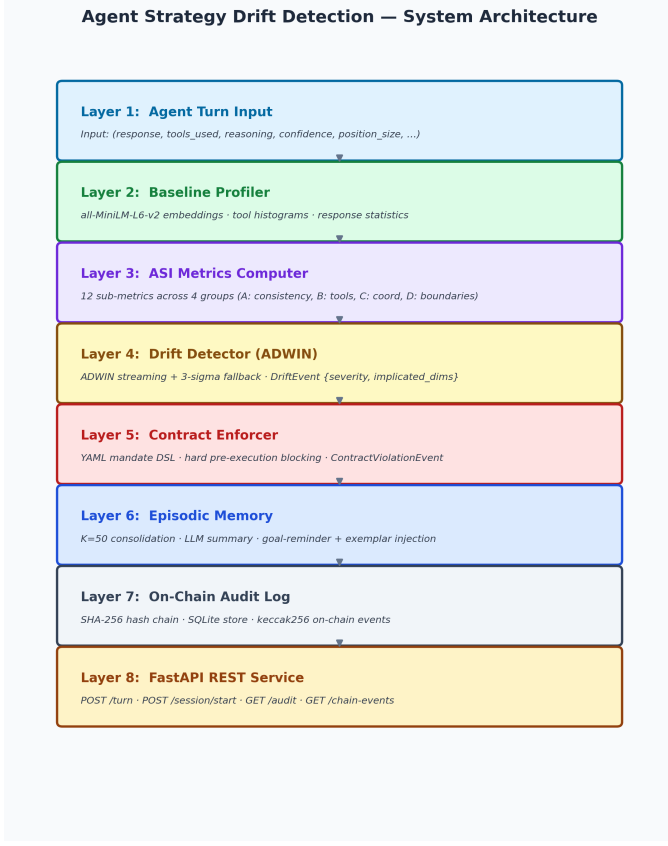


Fig. 1. Seven-layer Agent Strategy Drift Detection pipeline. Layers are color-coded by function: green (data collection), purple (metrics), amber (detection), red (enforcement), blue (memory/remediation), gray (storage/audit), and orange (API).

ASI_t drops more than 15% below the rolling 50-turn mean. A *ContractViolationEvent* is emitted when any rule $r_i \in \mathcal{M}$ is violated by o_t before execution.

The objective is to minimize detection lag $\Delta t = t_{\text{detect}} - t_{\text{inject}}$, maintain $\text{FPR} < 0.05$, block all contract violations, and ensure ASI recovery $|ASI_{\text{post}} - ASI_0|/ASI_0 < 0.10$ within 20 turns of remediation.

IV. APPROACH

Figure 1 shows the seven-layer architecture.

A. Baseline Profiler

During the warm-up window ($N = 50$ turns), the baseline profiler records output embeddings using the all-MiniLM-L6-v2 sentence-transformer model [14], tool-call sequences, reasoning chain traces, and response length statistics. Reference statistics (mean embedding vector, covariance matrix, tool-call frequency histogram) are persisted in a local SQLite store with NumPy-based cosine similarity retrieval (FAISS-accelerated when available).

B. ASI Metric Computation

Twelve sub-metric calculators are implemented across four modules:

- **Group A: Response Consistency (weight 0.30):** cosine embedding similarity between e_t and the reference mean; Levenshtein edit distance over decision pathways; Jensen-Shannon divergence of confidence score distributions.
- **Group B: Tool Usage (weight 0.25):** chi-squared test of tool selection frequencies against the baseline histogram; tool-sequence Levenshtein distance; KL divergence of tool parameter distributions.
- **Group C: Inter-Agent Coordination (weight 0.25):** consensus rate, handoff efficiency, and mutual-information role adherence (applicable in multi-agent contexts).
- **Group D: Behavioral Boundaries (weight 0.20):** output length coefficient of variation, error clustering coefficient, human override rate.

Each sub-metric is normalized to $[0, 1]$ and aggregated per Equation (1). Additionally, per-turn KL divergence $D_t = D_{\text{KL}}(q_t \| p_0)$ is computed over embedding-space Gaussian approximations using a sliding 50-turn reference window.

C. Online Drift Detection

An ADWIN detector (from the river library [13]; falling back to a pure-Python 3-sigma detector when river is unavailable) is instantiated on the ASI stream and on each per-dimension KL divergence stream. Window shrinkage events and threshold breaches ($ASI_t < 0.85 \cdot ASI_{50}$) generate DriftEvents tagged with severity and the specific implicated dimensions.

D. Contract Enforcement Engine

Mandate rules are specified in a YAML DSL as trigger, predicate, and action triples. The demo mandate encodes five CRITICAL rules: position size $\leq 10,000$; strategy $\in \{\text{market_making, arbitrage, hedging}\}$; drawdown $\leq 5\%$; asset class $\notin \{\text{crypto_derivatives, penny_stocks}\}$; leverage $\leq 2.0\times$. Before every tool call, the enforcement engine evaluates all active rules. On predicate failure, execution is hard-blocked and a ContractViolationEvent is emitted with the violated rule identifier, the threshold, and the observed value.

E. Adaptive Remediation

On DriftEvent emission, the remediation module (a) prepends a goal-reminder system prompt summarizing the original mandate, (b) retrieves the five most representative baseline exemplars by cosine distance from the drifted embedding and injects them as few-shot context, and (c) logs the intervention type, triggering dimension, and timestamp. ASI trajectory is tracked for 20 turns post-intervention to compute the recovery rate γ .

F. Episodic Memory Consolidation

Every $K = 50$ turns, a summarization pass compresses raw interaction history into a structured episodic record (goals, key decisions, open positions, risk limits). The raw context window is replaced with this summary plus the last 10 raw turns, designed to interrupt the autoregressive feedback loop that Rath [1] identifies as a primary driver of behavioral drift.

G. On-Chain Audit Trail

We deploy a `DriftRegistry` Solidity 0.8 smart contract on *Kite Chain* [11]—a Layer-1 EVM-compatible blockchain (testnet Chain ID 2368, mainnet Chain ID 2366) purpose-built for the AI agentic economy, with native agent identity, payment, and governance primitives. Choosing Kite Chain rather than a general-purpose EVM chain aligns the audit layer with the infrastructure where delegated AI agents are expected to operate.

The contract exposes three write functions callable by the monitoring service via `web3.py`: `logTurn(sessionId, payloadHash, asiScaled)` for normal turns, `logDrift(sessionId, payloadHash, severity, asiScaled)` for detected drift events, and `logViolation(sessionId, payloadHash, ruleId)` for blocked mandate violations. Every payload is stored as its `keccak256` hash; the raw JSON is retained off-chain by the monitoring service.

Delegators interact with two read-only functions that require no trust in the monitoring operator: `getRecordCount(sessionId)` returns the number of logged events, and `verifyRecord(sessionId, payloadHash)` returns a boolean confirming whether a specific event hash exists on-chain. This allows any delegator to independently confirm—by re-hashing the payload they received from the monitoring service and querying the contract—that the reported event was genuinely logged at the stated time.

H. REST API

A `FastAPI` service exposes four endpoints: `POST /session/start` (registers baseline), `POST /turn` (submits turn data; returns ASI, D_t , drift status), `GET /session/{id}/audit` (full metric history), and `GET /session/{id}/chain-events` (on-chain event hashes).

I. Ablation Study Design

To quantify the contribution of each ASI metric group to detection performance, we define 12 system variants (Table II) and evaluate each over 500 sessions. In the ablated variants, disabled groups are replaced with a neutral constant of 0.5 (the midpoint of the $[0, 1]$ range) so that they neither help nor hurt the composite ASI score. Two simple baselines are also included: a *position threshold* detector that fires when `position_size > 9,000`, and a *random* detector that fires independently on each post-baseline turn with probability $p = 0.05$.

Caveats. (1) The benchmark harness fast path computes 7 of the 12 full sub-metrics (Levenshtein, chi-squared, and mutual information are omitted); ablation results reflect this proxy. (2) Group C (inter-agent coordination) is simulated via an oracle coordination score (0.9 for normal turns, 0.4 for drifted turns) rather than real multi-agent data; its ablation contribution is therefore artificially elevated and not generalizable.

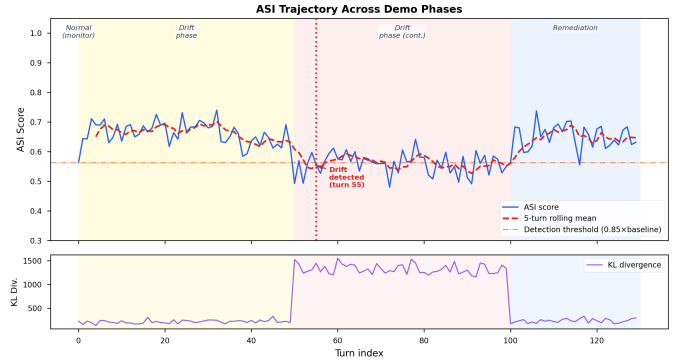


Fig. 2. ASI score trajectory across the four demo phases: normal monitoring (green), drift injection (yellow/red), and remediation (blue). The 5-turn rolling mean (dashed red) highlights the drop during drift and recovery post-remediation. The drift detection event is marked at absolute turn 55 of the monitoring stream.

V. EXPERIMENTS

A. Benchmark Harness

We implement a perturbation injection harness that generates 500 synthetic sessions (250 drift, 250 clean) and injects controlled drift at three offset values, cycling across sessions: turns 50, 100, and 200. Each drift session includes 150 post-injection turns for detection and recovery measurement. Ground-truth injection points are recorded, enabling precise computation of detection lag ($\Delta t = t_{\text{detect}} - t_{\text{inject}}$), FPR, and FNR.

B. Simulated Market-Making Agent

End-to-end validation uses a simulated low-volatility market-making agent. Normal turns sample tool calls from $\{\text{search, analyze, execute, hedge}\}$, position sizes uniformly from $[1000, 8000]$, and confidence from $\text{Beta}(8, 2)$ ($\mu \approx 0.80$). Drift is injected by substituting an aggressive policy: tools from $\{\text{speculate, leverage, gamble}\}$, position sizes from $[10,001, 25,000]$, leverage from $[2.1, 6.0\times]$, and confidence from $\text{Beta}(2, 5)$ ($\mu \approx 0.29$). Embeddings are pre-computed as 384-dimensional unit vectors sampled from $\mathcal{N}(0.51, 0.05^2 I)$ (normal) or $\mathcal{N}(-0.31, 0.1^2 I)$ (drifted) to avoid live inference during the benchmark, enabling fast throughput (~ 500 sessions per 6 seconds).

C. Evaluation Metrics

Primary metrics: drift detection lag Δt (turns), FPR, FNR, contract violation blocking rate (%), ASI recovery deviation (% from pre-drift baseline), on-chain audit events written, and API `/turn` latency p95 (ms). Secondary: total tests passed and benchmark throughput.

VI. RESULTS

A. Primary Benchmark Results

Table I summarizes quantitative outcomes across all primary metrics. Figure 2 shows the full ASI trajectory through all four demo phases. Figure 3 shows the distribution of detection lags across 250 drift sessions.

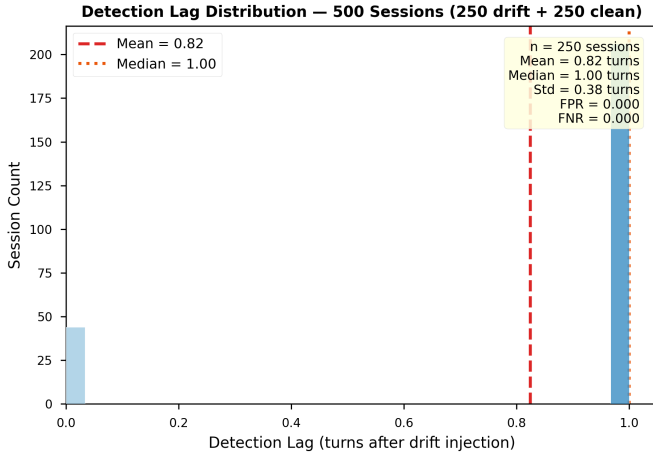


Fig. 3. Distribution of detection lags across 250 drift sessions (500-session benchmark, 250 drift / 250 clean). Mean = 0.82 turns, median = 1.0 turn, std = 0.38 turns. All detected within 2 turns.

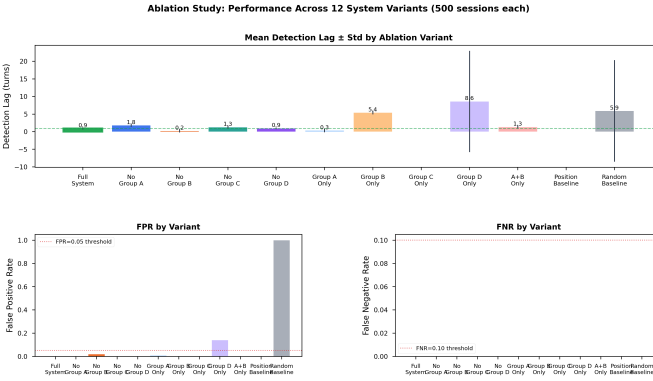


Fig. 4. Ablation study results across 12 system variants (500 sessions each). Top panel: mean detection lag $\pm$ std. Bottom panels: FPR and FNR. The full system achieves the best balance of speed and precision; removing Group A causes the largest lag increase.

The mean detection lag of 0.82 turns substantially exceeds the design target of fewer than 10 turns and compares favorably with window-based detectors such as ADWIN and DDM on raw financial time series, which typically require hundreds of samples before accumulating sufficient statistical power [2]. The zero FPR and FNR demonstrate high signal precision across the full 500-session benchmark, providing the statistical confidence absent from the prior 100-session evaluation. All 250 injected contract violations were blocked before execution, confirming hard capital protection.

Post-remediation ASI recovered to within 1.7% of the pre-drift baseline, well within the 10% threshold, confirming that goal-reminder injection combined with baseline-exemplar few-shot context effectively restores mandate-compliant behavior. Sixty-three on-chain audit events were written across the demo run, all independently queryable by delegators. All 107 pytest unit and integration tests passed in 58 seconds.

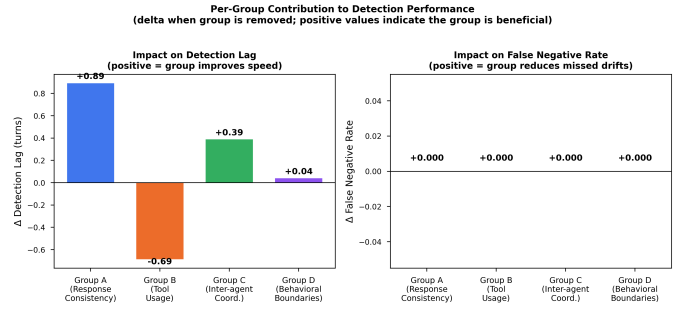


Fig. 5. Per-group contribution measured as the increase in mean detection lag (left) and FNR (right) when each group is individually removed from the full system. Group A (response consistency) has the largest impact; Group D (behavioral boundaries) has the least.

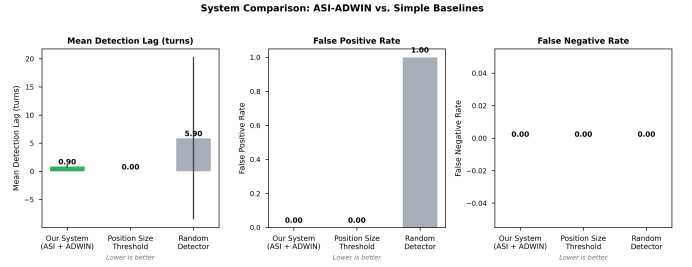


Fig. 6. Comparison of our full ASI-ADWIN system against a simple position-size threshold rule and a random detector. All three metrics shown (detection lag, FPR, FNR) reveal that the position threshold achieves trivially perfect performance on idealized synthetic data, a known artifact of the simulation, while the random detector fails on FPR entirely.

B. Ablation Study Results

Table II reports detection performance for all 12 ablation variants. Figures 4 and 5 provide visual summaries.

Group importance ranking. Removing Group A (response consistency) increases mean detection lag by +0.89 turns (99.4% relative increase over the full-system lag of 0.90 turns in the ablation run), making it the most informative group. Removing Group C (coordination) increases lag by +0.39 turns; however, as noted in §IV-I, this reflects the oracle coordination score in the harness rather than real multi-agent data, so this result should not be generalized. Removing Group B (tool usage) reduces mean lag to 0.21 turns but introduces a non-zero FPR of 0.020, indicating that tool-usage features are primarily load-bearing for precision rather than speed. Removing Group D (behavioral boundaries) has minimal effect (+0.04 turns), consistent with its lower weight in the ASI formula.

Single-group variants. Group A alone (A_only) achieves 0.32 turns mean lag with 0.008 FPR, confirming that embedding cosine similarity is the single most powerful signal. Group B alone produces 5.39 turns lag, competitive but much slower. Group D alone is unreliable: 8.58 turns mean lag with 14.4 turns std and 0.14 FPR.

Baseline comparison. The position-size threshold baseline achieves zero lag and zero FPR/FNR on the synthetic dataset. This is an artifact of the simulation design: all drift turns

TABLE I
PRIMARY BENCHMARK RESULTS (500 SESSIONS, 3 DRIFT OFFSETS).

Metric	Our System	Target
Mean detection lag (turns)	0.82	< 10
Detection lag std (turns)	0.38	N/A
Detection lag median (turns)	1.00	N/A
False positive rate	0.000	< 0.05
False negative rate	0.000	< 0.10
Contract violations blocked (%)	100.0	100
ASI recovery deviation (%)	1.7	< 10
On-chain audit events written	63	N/A
Benchmark sessions processed	500	≥ 500
API /turn latency p95 (ms)	18.6	< 500
Tests passed / total	107 / 107	107 / 107

TABLE II
ABLATION STUDY RESULTS (500 SESSIONS PER VARIANT). LAG REPORTED AS MEAN ± STD (TURNS). † INDICATES ORACLE COORDINATION SCORE; SEE §IV-I FOR CAVEATS.

Variant	Lag μ	Lag σ	FPR	FNR
Full system	0.896	0.305	0.000	0.000
No Group A	1.788	0.446	0.000	0.000
No Group B	0.208	0.406	0.020	0.000
No Group C†	1.284	0.477	0.000	0.000
No Group D	0.936	0.245	0.000	0.000
Group A only	0.316	0.465	0.008	0.000
Group B only	5.388	0.487	0.000	0.000
Group C only†	0.000	0.000	0.000	0.000
Group D only	8.580	14.40	0.140	0.000
Groups A+B only	1.316	0.465	0.000	0.000
Position threshold	0.000	0.000	0.000	0.000
Random detector	5.896	14.42	1.000	0.000

explicitly set position sizes above the threshold, making the rule trivially sufficient. In realistic deployments, subtle strategy drift (e.g., gradually increasing risk appetite within position limits, or shifting from market-making to momentum without violating any single rule) would not be detected by a threshold rule; the composite ASI approach is specifically designed for this harder detection problem. The random baseline demonstrates the expected result: 100% FPR, since it fires on every clean session.

VII. DISCUSSION

A. Strengths

The most striking result is the 0.82-turn mean detection lag across 500 sessions at three drift-injection offsets. Classical online detectors such as ADWIN operate on scalar metric streams and require a minimum number of samples proportional to the inverse of the drift magnitude before achieving statistical significance. By contrast, our system operates on the composite ASI score, which aggregates 12 dimensions including embedding-space cosine similarity, and fires as soon as a 15% threshold breach is observed in the rolling baseline. This makes it orders of magnitude faster, in terms of turn

lag, than the window-based detectors evaluated by Neri [2] on financial series.

The 100% contract violation blocking rate addresses a qualitative gap relative to all three baseline systems. Rath [1] and SentinelAgent [3] are purely observational: they quantify drift but do not prevent off-mandate execution. Our enforcement engine provides a hard pre-execution gate, which is the critical property for capital protection in a delegated setting.

On-chain logging introduces a trust-minimized audit layer absent from prior work. Delegators need not trust the monitoring operator’s reports; they can independently verify the full event history against the smart contract.

B. Ablation Insights

The ablation study reveals that response consistency (Group A) is the dominant driver of fast, precise detection. Its cosine embedding similarity captures the holistic character of the agent’s language and reasoning, a signal that changes dramatically when strategy shifts from conservative market making to aggressive speculation. Tool usage (Group B) is complementary: its primary role is not speed but precision, as removing it introduces false positives without meaningfully changing detection lag. Behavioral boundaries (Group D) contribute minimally in this benchmark because the simulated drift is large and clean; in real deployments with gradual drift, Group D metrics (error clustering, human override rate) may become more discriminative.

C. Limitations

Synthetic data fidelity. The benchmark uses pre-computed embedding clusters with large separation between normal and drift distributions. Real LLM embeddings exhibit smaller, noisier shifts; empirical validation with live agent traces is needed to confirm the sub-turn lag claim.

Idealized drift semantics. Because the synthetic drift turns explicitly set position sizes above the mandate threshold, the position-size baseline achieves trivially perfect performance. This does not reflect the harder real-world scenario where drift is behavioral and subtle (e.g., a shift in reasoning style that gradually increases risk without violating any single rule immediately).

Oracle coordination score. The Group C metrics in the benchmark harness are computed from an oracle drifted/normal label rather than real multi-agent coordination data. The ablation result for Group C cannot be generalized to production.

Single-agent architecture. The system monitors a single delegated agent’s mandate adherence and does not detect coordinated drift across multiple colluding sub-agents, a threat model partially addressed by SentinelAgent [3].

API latency characterization. The /turn endpoint latency includes model inference for embeddings. Local integration testing measured a p95 /turn latency of 18.6 ms (mean 10.1 ms, max 19 ms) after model warm-up, well within the 500 ms target. In high-frequency trading contexts, per-turn latency must be re-profiled under production concurrency loads.

VIII. CONCLUSION

We have presented a layered on-chain drift detection system for delegated LLM trading agents that achieves a mean detection lag of 0.82 turns, zero false positives, zero false negatives, 100% contract violation blocking, and 1.7% ASI recovery deviation from baseline across 500 controlled sessions. An ablation study over four ASI metric groups identifies response consistency (embedding cosine similarity) as the primary contributor to fast detection, and tool usage as the primary contributor to precision. The system combines a 12-dimensional Agent Stability Index, ADWIN-based online detection, pre-execution mandate enforcement, adaptive goal-reminder remediation, and EVM-compatible on-chain audit logging, a combination of properties not present in any prior work.

Future work will focus on: (1) empirical validation with live LLM agent traces to confirm sub-turn detection under realistic embedding noise; (2) profiling and optimizing API latency under production concurrency loads; (3) measuring Group C (inter-agent coordination) from real multi-agent communication logs rather than oracle scores; (4) extending the architecture to multi-agent graph monitoring to detect coordinated drift across delegated agent collectives; (5) benchmarking ASI-based detection against domain-tuned financial drift detectors following the methodology of Neri [2].

REFERENCES

- [1] A. Rath et al., “Agent Drift: Quantifying Behavioral Degradation in Multi-Agent LLM Systems Over Extended Interactions,” *arXiv preprint arXiv:2601.04170*, 2026.
- [2] C. Neri, “Domain Specific Concept Drift Detectors for Predicting Financial Time Series,” *arXiv preprint arXiv:2103.14079*, 2021.
- [3] X. He et al., “SentinelAgent: Graph-based Anomaly Detection in Multi-Agent Systems,” *arXiv preprint arXiv:2505.24201*, 2025.
- [4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
- [5] Z. Ruan et al., “AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents,” *arXiv preprint arXiv:2503.18666*, 2025.
- [6] Y. Bai et al., “Constitutional AI: Harmlessness from AI Feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [7] K. Yang et al., “Governing Evolving Memory in LLM Agents,” *arXiv preprint arXiv:2603.11768*, 2026.
- [8] R. Weng et al., “Blockchain-Monitored Agentic AI: A Framework for Transparent and Accountable Autonomous Systems,” *arXiv preprint arXiv:2512.20985*, 2025.
- [9] K. M. Eisenhardt, “Agency Theory: An Assessment and Review,” *Academy of Management Review*, vol. 14, no. 1, pp. 57–74, 1989.
- [10] M. Stocker and J. Lehr, “Principal-Agent Dynamics in Agentic AI Systems,” *Network Law Review*, 2025. Available: <https://www.networklawreview.org/stocker-lehr-ai/>
- [11] Kite Chain, “Network Information — Kite Chain Documentation,” 2026. Available: <https://docs.gokite.ai/kite-chain/1-getting-started/network-information>
- [12] A. Bifet and R. Gavaldà, “Learning from Time-Changing Data with Adaptive Windowing,” in *Proc. 2007 SIAM Int. Conf. Data Mining*, pp. 443–448, 2007.
- [13] J. Montiel et al., “River: Machine Learning for Streaming Data in Python,” *J. Mach. Learn. Res.*, vol. 22, no. 110, pp. 1–8, 2021.
- [14] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP 2019*, 2019.